Fuzzing

Description

Tecnica di **testing automatico** utilizzata per scoprire vulnerabilità nei programmi. Funziona generando e inviando **input casuali o malformati** a un'applicazione per individuare comportamenti anomali, come crash, memory leaks o violazioni di sicurezza.

Come funziona il fuzzing?

1. **Generazione di input** → Il fuzzer crea dati casuali, mutati o strutturati.
2. **Esecuzione del target** → Il programma viene eseguito con questi input.
3. **Monitoraggio dei risultati** → Si osservano crash, errori o anomalie.
4. **Analisi e debugging** → Se viene trovato un problema, si analizza il codice per identificarne la causa.



I tool di fuzzing sono il risultato della composizione di tre moduli:

- Il generatore di dati malformati
- Il distributore dei dati verso il modulo target
- Un component di monitoraggio per gestire eventuali errori e scoprire il percorso del dato in input nel modulo

Application fuzzing
Testa funzioni quali i pulsanti e i campi di immissione di programmi grafici o le opzioni dei programmi della riga di comando

Protocol fuzzing
Verifica il comportamento del programma in caso di invio di contenuti con formato errato. In particolare, i contenuti inviati non devono essere interpretati come comandi ed eseguiti.

File format fuzzing
Genera file errati da elaborare. Possono determinarsi problemi nel caso in cui il file non presenta il formato atteso. Si testano anche le funzioni, ad esempio la compressione dei file

American Fuzz Lop
Può compilare autonomamente il codice sorgente e testarlo; Se il testo sorgente non è disponibile, viene eseguita un'emulazione con QEMU (Quick Emulator)

Fuzzino
Offre una libreria per la generazione di dati di input che possono essere utilizzati per testare le applicazioni (basate su protocollo).

LibFuzzer
Libreria che fa parte dell'infrastruttura di compilazione LLVM.

ClusterFuzz
Ambiente di test originariamente sviluppato da Google per-testare il browser Chrome.

Sulley
Offre una raccolta di strumenti nel linguaggio di programmazione Python; adatto in particolare per eseguire test semplici, come la generazione di dati casuali.

Peach
Soluzione fortemente automatizzata per il fuzz testing di hardware e software.

Powerfuzzer
Offre diversi scenari di attacco, come iniezioni SQL; è
utilizzato attraverso un'interfaccia utente grafica basata sul web.

Classificazione fuzzer

Black-box Fuzzing

- Non ha conoscenza interna del codice sorgente o del comportamento dell'applicazione.
- Investe sulla quantità di test piuttosto che sulla qualità.
- **Esempio:** *Boofuzz* (fuzzing di protocolli di rete).

White-box Fuzzing

- Ha accesso al codice sorgente e usa tecniche come **symbolic execution** e **analisi statica** per generare input mirati.
- Più efficace ma anche più complesso da implementare.
- **Esempio:** SAGE (Microsoft).

Grey-box Fuzzing

- Conosce alcune informazioni sul software testato, come la struttura del codice o i dati d'ingresso accettabili.
- Spesso utilizza **strumenti di code coverage** per ottimizzare i test.
- **Esempio:** AFL (American Fuzzy Lop).

In base alla generazione dell'input

Generative Fuzzing

- Crea input **da zero** secondo un modello o un formato predefinito (es. XML, JSON, protocolli di rete).
- Utile per testare parser o applicazioni che accettano input strutturati.
- **Esempio:** Peach Fuzzer.

Mutation-based Fuzzing

- Parte da input validi e li **modifica casualmente** (mutazioni bit-flip, byte-swap, ecc.).
- Più leggero e veloce perché non ha bisogno di conoscere il formato dell'input.
- **Esempio:** AFL, Radamsa.

In base al livello di automazione

Coverage-guided Fuzzing

- Analizza la copertura del codice per ottimizzare gli input e trovare più bug con meno test.
- Usa tecniche di **instrumentation** per monitorare il codice in esecuzione.
- **Esempio:** AFL, LibFuzzer.

Dumb Fuzzing (Random Fuzzing)

- Genera input casuali senza alcuna logica o conoscenza del target.
- Molto veloce ma meno efficace rispetto ad altre tecniche.
- **Esempio:** zzuf.

Tainting analysis

Tecnica di **analisi statica o dinamica** usata per tracciare il flusso di dati potenzialmente non sicuri all'interno di un programma.

È molto usata nella **cybersecurity** per individuare vulnerabilità come **SQL Injection, Cross-Site Scripting (XSS), buffer overflow** e altre forme di attacchi.

1. **Identificazione delle fonti (sources)** → Dati non fidati, come input dell'utente, variabili di ambiente, dati di rete, ecc.
2. **Propagazione della contaminazione** → Se una variabile dipende da un dato contaminato, viene marcata anch'essa come contaminata.
3. **Individuazione dei sink (destinazioni pericolose)** → Funzioni critiche come `exec()`, `eval()`, query SQL o scrittura su file.
4. **Verifica della sanificazione** → Se il dato contaminato arriva a un sink senza essere filtrato o validato, viene segnalata una potenziale vulnerabilità.

Esempio

Immaginiamo il seguente codice in Python:

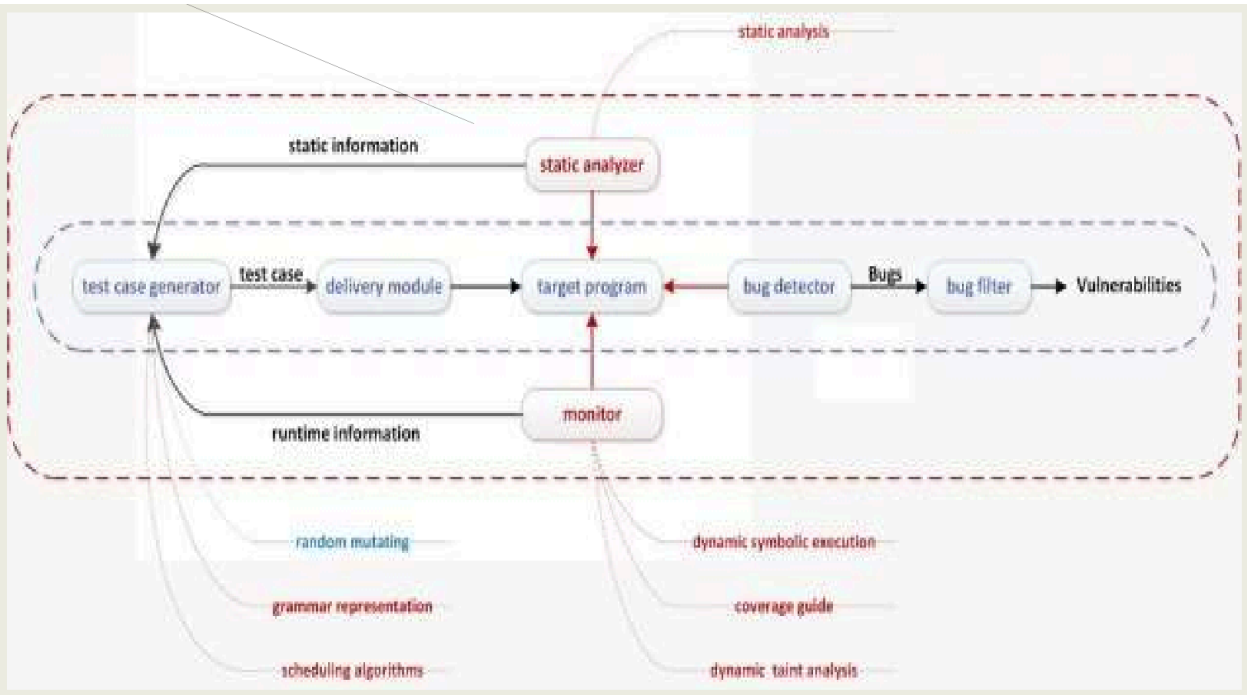
```
import sqlite3

def login(user_input):
    query = f"SELECT * FROM users WHERE username = '{user_inp
ut}'"
    db.execute(query)  # 🚨 Potenziale SQL Injection!
```

- Se `user_input` proviene da un utente, viene marcato come **tainted** (contaminato).
- Poiché il valore finisce direttamente in una query SQL senza validazione, viene **propagato**.
- Il sink è `db.execute()`, che esegue query SQL. Se la sanitizzazione è assente, si segnala una vulnerabilità.

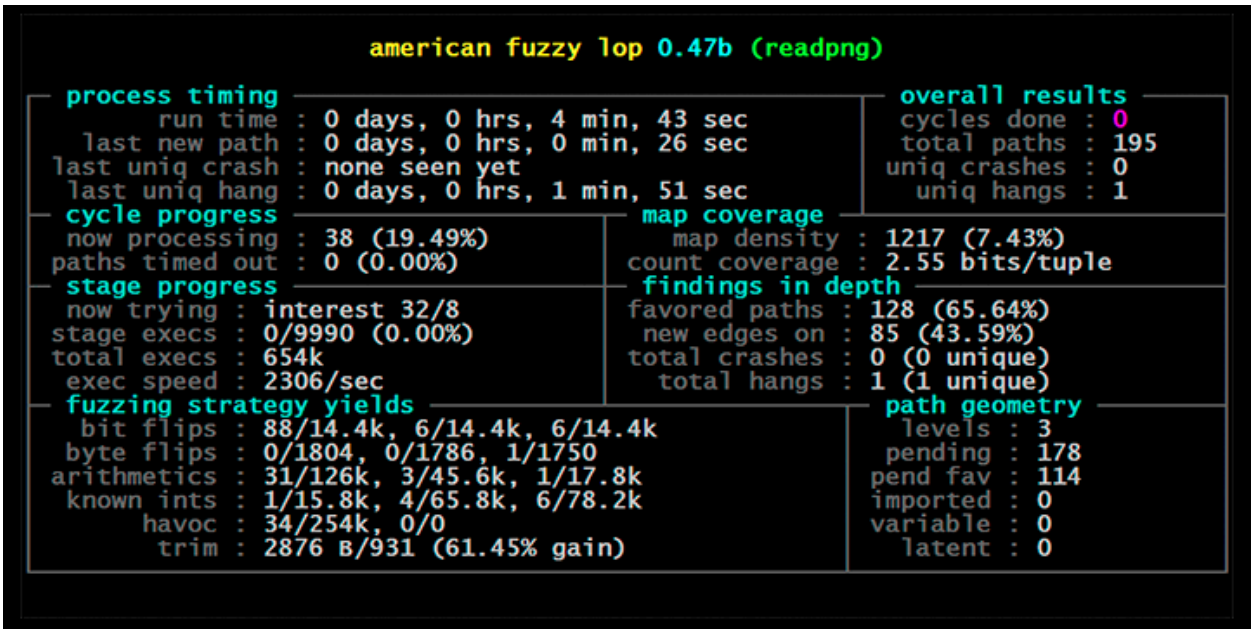
Una soluzione sicura sarebbe usare **query parametrizzate**, che rimuovono la contaminazione:

```
query = "SELECT * FROM users WHERE username = ?"
db.execute(query, (user_input,))
```

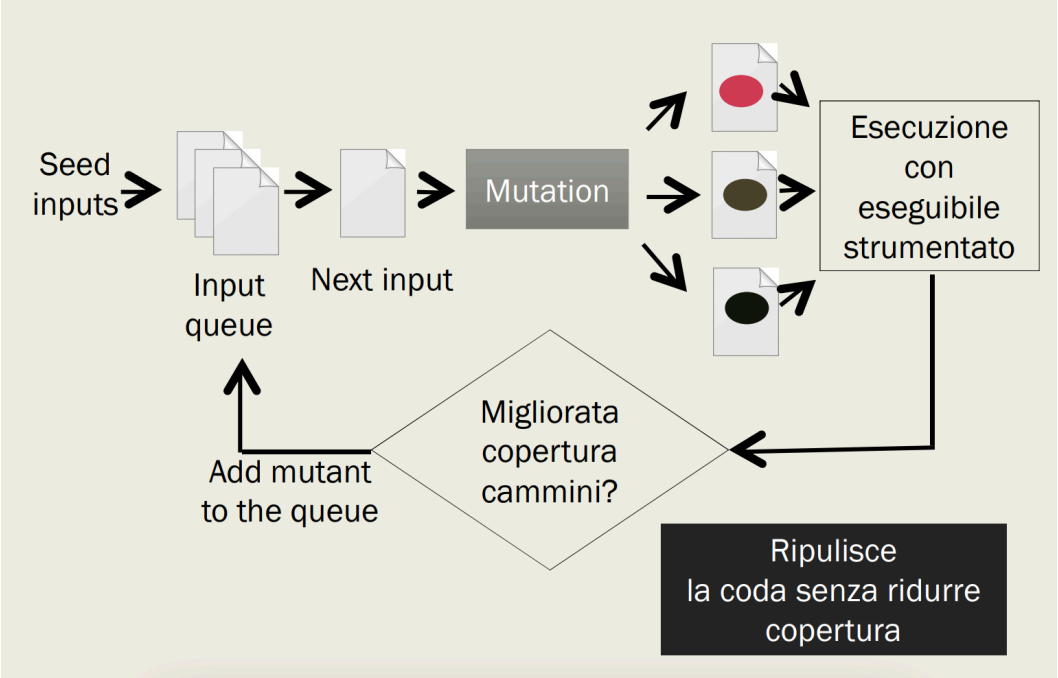


American Fuzzy Lop (AFL)

American Fuzzy Lop (AFL) è un fuzzer open source molto popolare, sviluppato che ha rivoluzionato il modo in cui vengono condotti i test di sicurezza sui software. Ecco alcuni punti chiave:



- ***Coverage-Guided Fuzzing***: AFL utilizza un approccio guidato dalla copertura del codice. Durante l'esecuzione, raccoglie informazioni sui percorsi eseguiti dal target e, in base a questi dati, genera nuovi input mirati a esplorare parti non coperte precedentemente.
- ***Mutation-Based Fuzzing***: Parte da input validi (seed) e li modifica (mutazioni) per generare varianti che possano innescare comportamenti inaspettati, come crash o comportamenti anomali.
- ***Efficienza ed Efficacia***: Grazie al suo approccio innovativo, AFL è in grado di trovare vulnerabilità e bug in modo rapido ed efficiente, ed è stato ampiamente utilizzato per testare applicazioni di vario tipo.
- ***Facilità di Utilizzo***: Pur essendo uno strumento molto potente, AFL è relativamente semplice da configurare e utilizzare, soprattutto su applicazioni compilate con opportune tecniche di instrumentation.
- ***Ampia Adozione***: È largamente adottato sia nella comunità della cybersecurity che nello sviluppo software, essendo in grado di testare applicazioni in linguaggi come C/C++ e molti altri.



Utilizzo dell'AFL

1. Preparazione dell'ambiente

- **Installazione:**

  Assicurati di avere installato AFL sul tuo sistema. Su Linux puoi spesso installarlo dai repository o compilarlo dal sorgente.

- **Compilazione del target:**

  AFL necessita che il binario da testare sia compilato con la sua instrumentation. Utilizza i compilatori forniti da AFL (ad esempio `afl-clang-fast` o `afl-gcc`) per compilare il tuo codice.

  ```
  afl-clang-fast -o target_program target_program.c
  ```

  In questo modo, il binario includerà il supporto per il monitoring della copertura del codice.

---

2. Preparazione dei file seed

- **Directory di input:**Crea una directory (ad esempio `input/`) e inserisci al suo interno alcuni file di esempio (seed) che rappresentino input validi o parzialmente validi per il tuo programma. Questi file aiuteranno AFL a partire da casi noti.

```
mkdir input
echo "test" > input/test.txt
```

---

3. Avvio del fuzzing

- **Comando base:**Esegui AFL specificando la directory degli input (`i`), la directory per salvare i risultati (`o`), e il comando per eseguire il target. Utilizza il placeholder `@@` per indicare ad AFL dove inserire il file di input.In questo modo, AFL inizierà a mutare i file seed e a eseguire il programma con gli input generati, monitorando la copertura e i crash.

```
afl-fuzz -i input/ -o output/ -- ./target_program @@
```

---

4. Monitoraggio e analisi

- **Dashboard interattiva:**

  AFL fornisce una dashboard in tempo reale che mostra statistiche, come il numero di esecuzioni, i casi interessanti trovati e eventuali crash.

- **Analisi dei crash:**

  Se vengono trovate anomalie, AFL salverà i file che hanno causato il crash nella directory di output (solitamente in una cartella come `crashes/`). Puoi analizzare questi file per capire il problema.

---

5. Suggerimenti utili

- **Ottimizzazione dei seed:**

  Se i file seed sono troppo semplici, il fuzzing potrebbe richiedere più tempo per esplorare scenari complessi. Prova a migliorare i seed per ottenere input più vari e rappresentativi.

- **Monitoraggio delle performance:**

  AFL può richiedere molte risorse, quindi assicurati di monitorare l'utilizzo della CPU e della memoria, specialmente su applicazioni più complesse.

- **Uso di opzioni avanzate:**

  AFL offre molte opzioni per personalizzare il fuzzing (ad es. timeout, limiti di memoria, ecc.). Consulta la documentazione ufficiale per approfondire:

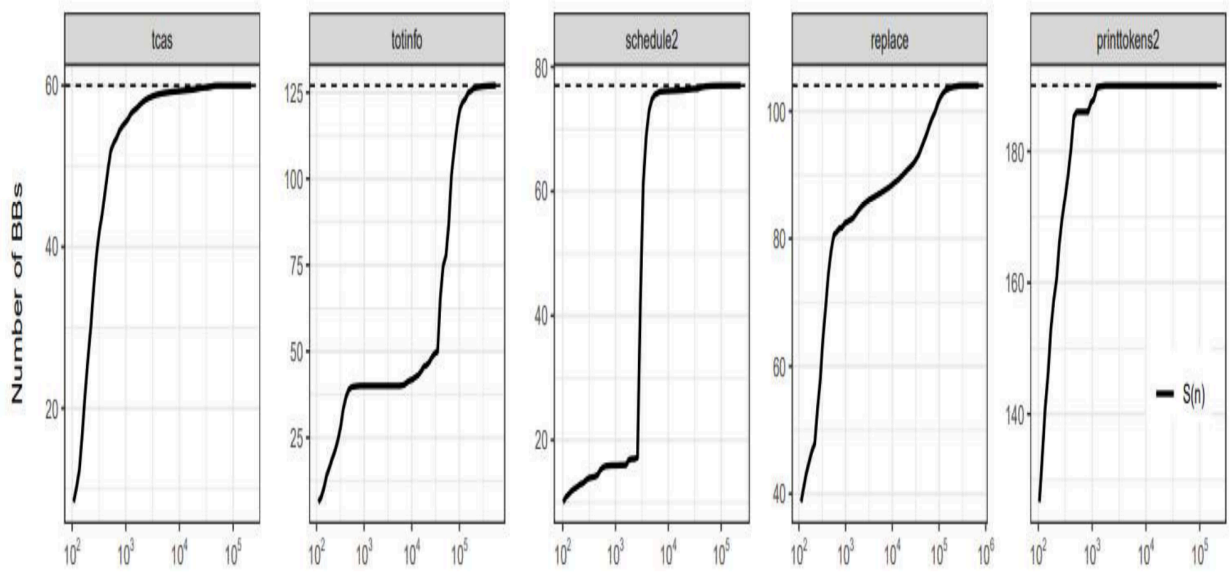
```
afl-fuzz -h
```

Bug/Vulnerabilità che può scoprire

- **Memory Leaks**
  Incorrect management of memory allocation, which is exhausting available memory
- **Injections**
  Untrusted input supplied to a program
- **Sensitive Data Exposure**
  When an application inadvertently exposes personal data
- **Insecure Deserialization**
  Manipulated object injected into the context of a web application
- **Buffer Overflow**
  Exceeding the buffer when writing data, which can lead to overwriting adjacent memory locations
- **Use After Free**
  No pointing to a valid object that can lead to data corruption, segmentation faults, or general protection faults
- **Data Races**
  Accessing the same memory location concurrently in multi-thread process, which can lead to security vulnerabilities
- **Software Crashes**
  Failing to function properly and exiting, possible fatal system errors
- **Functional Bugs**
  Ceasing to respond to inputs
- **Uncaught Exceptions**
  Not properly handling exceptions, which can lead to disrupting program executing
- **Undefined Behavior**
  Executing code not prescribed by language specification (C/C++), which can lead to security vulnerabilities
- **And many more**

Technique	Effort	Code coverage	Defects Found
black box + mutation	10 min	50%	25%
black box + generation	30 min	80%	50%
white box + mutation	2 hours	80%	50%
white box + generation	2.5 hours	99%	100%

Alcuni risultati



Time to first failure (TTFF)

To measure the relative maturity of a given protocol, the time to first failure (TTFF), or the time to the first instance when that a protocol crashes, is used. It

The average time for first failure of all tests in 2016 was about 1 hour (1.4 hours). This represents a very slight decrease over 2015 (1.7 hours), which might be attributable to a more diverse range of tests run in 2016.

5 most mature protocols (average time to first failure, in hours and days)

Protocol suite	2016 time to first failure	2015 time to first failure	2016 test runtime (avg)
TLS Client (Core IP)	9.6 hours	5.4 hours	10.5 hours
SSH2 (Core IP)	6.8 hours	1.9 hours	22.6 hours
SSH Server (Core IP)	4.9 hours	0.7 hours	5.96 hours
802.11 (Core IP)	4.2 hours	1 hour	11.8 hours
ICMPv6 (Core IP)	2.7 hours	1 hour	2.3 days

5 least mature protocols (average time to first failure, in minutes and seconds)

Protocol suite	2016 time to first failure	2015 time to first failure	2016 test runtime (avg)
IEC-61850 MMS (ICS)	6.6 seconds	13.2 seconds	5.7 hours
MODBUS PLC (ICS)	1.8 minutes	6 seconds	35 minutes
SNMP Trap (Remote Management)	1.8 minutes	3.6 minutes	57 minutes
MQTT (ICS)	9.9 minutes	2.1 minutes	1.3 hours
DNP3 (ICS)	14 minutes	3.6 minutes	2.6 hours

	Types	Test cases	Time (hours)	Instrumentation	Allowed failures	Test harness integration	Processes	Documentation	Other requirements
5 : Optimized	G T	infinite infinite	720	O,A,D	none	Yes	S,C	RR,S,P,B,C	U2
4 : Integrated	G T	infinite infinite	168	O,A,D	none		S,C	RR,S,P,B,C	
3 : Managed	G T	2,000,000 5,000,000	16	O,A	Tr		S	RR,S,P,B	
2 : Defined	G (T)	1,000,000 (5,000,000)	8	O	Tr		S	RR,S,P	
1 : Initial	G/T	100,000	2	O	Tr,As			RR	
0 : Immature									

G = Generational fuzzer
T = Template fuzzer
O = Human observation
A = Automated instrumentation
D = Developer tooling
Tr = Transient failures
As = Non-DoS assertion failures

S = Attack surface analysis
C = Software component analysis
RR = Results and summary
P = Test plan
B = Document baseline test configuration
U2 = Use two different fuzzers per type

Standard che raccomandano fuzzing

ISO 26262
Road vehicles – Functional Safety

UNECE WP.29
United Nations World Forum for Harmonization of Vehicle Regulations

ISA/IEC 62443-4-1
Secure Product Development Lifecycle Requirements

ISO/SAE DIS 21434
Road Vehicles — Cybersecurity Engineering

UL2900-1 and **UL2900-2-1**
Healthcare and Wellness Systems - Software Cybersecurity for Network-Connectable Products

ISO/IEC/IEEE 29119
Software and Systems Engineering - Software Testing

ISO/IEC 12207
Systems and Software Engineering – Software Life Cycle Processes

ISO 27001
Information Technology – Security Techniques – Information Security Management Systems

ISO 22301
Security and Resilience — Business Continuity Management Systems

IT-Grundschutz (Germany)
Based on ISO 27001

NIST SP 800-95
Web Services — standard for software testing (USA) and others